ROOMRENT PROJECT ARCHITECTURE

RoomRent is a Flask web app for browsing rental listings behind login-protected pages. It combines server-rendered Jinja templates with client-side JavaScript that fetches JSON APIs and fills listing/property views.

TOP-LEVEL STRUCTURE

app.py: Flask application, routes, authentication flow, API endpoints, inquiry form, and app startup.

auth_store.py: local JSON-backed user storage for phone OTP users and Google OAuth profiles.

db.py: data access layer for MySQL with a demo-data fallback.

templates/: Jinja HTML templates for shared layout, home, auth pages, property detail, and 404.

static/css/styles.css: all styling and responsive layout rules.

static/js/app.js: browser-side API loading, rendering, and small UI behaviors.

.env: local runtime configuration and secrets, ignored by Git.

instance/users.json: runtime user data created by auth_store.py, ignored by Git.

RUNTIME DIAGRAM

Browser -> Flask routes in app.py -> Jinja templates in templates/ -> CSS/JS from static/.

Browser JavaScript -> /api/storage, /api/listings, /api/listings/<id> -> app.py -> db.py -> MySQL or SAMPLE_LISTINGS.

Auth routes in app.py -> auth_store.py -> instance/users.json.

Inquiry form -> /inquiry -> db.py -> MySQL inquiries table or DEMO_INQUIRIES memory list.

AUTHENTICATION ARCHITECTURE

Phone login/signup starts in /login or /signup and calls _start_phone_otp.

The OTP is generated in app.py, hashed with Werkzeug, and stored in the Flask session with an expiry timestamp.

In local demo mode, the OTP is also flashed/shown to the user instead of being sent by SMS.

/verify-otp checks expiry and hash, then creates or updates the user through auth_store.py.

Google sign-in is optional. If Authlib and Google credentials exist, /auth/google and /auth/google/callback handle OAuth and upsert a Google user.

After successful auth, app.py stores user_id in the session and templates receive current_user from the context processor.

DATA ARCHITECTURE

db.py first tries MySQL when MYSQL_* or DB_* configuration is present and mysql.connector is installed.

If MySQL is unavailable, listing search and detail pages use SAMPLE_LISTINGS from db.py.

Listing APIs return JSON. app.js renders listing cards and property details in the browser.

Inquiries are inserted into a MySQL inquiries table when possible. In demo mode they are appended to DEMO_INQUIRIES and are not persistent.

FRONTEND ARCHITECTURE

base.html provides the shared shell: header, profile popover, nav, flash messages, main block, footer, CSS, and JS.

index.html provides the search form and an initially hidden listing grid. app.js loads /api/listings and renders cards.

property.html provides data-* placeholders. app.js loads /api/listings/<id> and fills the hero, details, amenities, and inquiry form.

login.html, signup.html, and verify_otp.html are server-rendered auth forms.

styles.css owns all page layouts and responsive behavior. app.js owns dynamic data rendering and lightweight interaction.

ROUTE MAP

GET/POST /login: phone login form and OTP start.

GET/POST /signup: phone signup form and OTP start.

GET/POST /verify-otp: OTP verification and user login.

GET /auth/google: start Google OAuth.

GET /auth/google/callback: finish Google OAuth.

GET /logout: clear session.

GET /: protected home/search page.

GET /property/<listing_id>: protected property detail shell.

GET /api/storage: current data source label.

GET /api/listings: filtered listing JSON.

GET /api/listings/<listing_id>: single listing JSON.

POST /inquiry: save or demo-store owner inquiry.

IMPORTANT LIMITS

auth_store.py uses a JSON file and has no locking, so it is best for demos or low-concurrency local use.

Phone OTP production delivery is not implemented; OTP_DEMO_MODE=0 requires adding an SMS provider integration first.

MySQL errors silently fall back to demo data in several paths, which is convenient locally but can hide production database problems.

.env contains secrets and should stay ignored by Git. Rotate credentials if they have been exposed outside your machine.